

1. 괄호 추가하기

문제 파악

1. 수식은 사칙연산과 1자리 자연수로만 이루어져 있다.
2. 괄호가 있을 경우에는 괄호 먼저, 괄호가 없으면 왼쪽부터 계산한다.
3. 괄호는 연속으로 넣을 수 없으며, 중첩될 수 없고, 1개의 연산자만을 감쌀 수 있다.

1. 괄호 추가하기

문제 풀이

불가능한 경우들:

$$(7 + 3 * 9) - 2$$

$$(7 + (3 * 9)) - 2$$

1. 괄호 추가하기

문제 풀이

왼쪽에서부터 무조건 연산한다는 규칙과 괄호 중첩이 불가능해서 괄호를 전부 먼저 계산할 필요가 없다. 괄호 간에 무조건 연산자 하나 이상이 존재하기 때문이다.

예시) $7 + (3 * 9) - (2 + 6)$

$(3 * 9)$ 와 $(2 + 6)$ 을 먼저 계산한 후에 나머지를 계산해도 되지만

$7 + (3 * 9) \quad | \quad - (2 + 6)$

$(3 * 9)$ 를 계산하고 7을 더한 후, 뒤의 $(2 + 6)$ 과 $-$ 를 계산해도 된다.

1. 괄호 추가하기

문제 풀이

재귀함수를 이용하여 괄호의 위치를 변경하면서 계산을 할 수 있다.

왼쪽에서부터 연산자를 정해서 계산을 하고 다음으로 넘어가던지,
이 연산자를 건너뛰고 바로 다음 연산자에 괄호를 씌우고 이 연산자를
계산하던지 두 개의 분기를 정하면 된다.

종료 조건은 마지막 연산자를 사용한 이후.

1. 괄호 추가하기

문제 풀이

예시) $7 + 3 * 9 - 2 + 6$

연산자 vector에 $+$, $*$, $-$, $+$ 가 차례로 들어있다. 시작 연산자는 $+$.

첫 번째 연산자를 계산하면 $10 * 9 - 2 + 6$ 이 되고, 재귀한다.

첫 번째 연산자를 계산하지 않고 바로 다음에 괄호를 씌우면
 $7 + (3 * 9) - 2 + 6 = 7 + 27 - 2 + 6$ 이 되고, 이때 괄호 바로 왼쪽의
현재 연산자 $+$ 를 계산해주면 $34 - 2 + 6$ 이 되고, 재귀한다.

1. 괄호 추가하기

핵심 코드

```
// 괄호 안에는 하나의 연산자만이 들어갈 수 있음
void whole(int pos, int sum) { // 연산자 위치, 현재까지 계산된 값
    if (pos >= op.size()) { // 남은 연산자 전부 소진
        max_sum = max(max_sum, sum);
        return;
    }

    whole(pos + 1, solve(sum, op[pos], num[pos + 1])); // 현재 위치 바로 계산

    if (pos + 2 <= op.size()) { // 현재 위치 다음의 연산자에 우선순위 주기
        int temp = solve(num[pos + 1], op[pos + 1], num[pos + 2]); // pos + 1 위치에 괄호를 넣어서 먼저 계산
        whole(pos + 2, solve(sum, op[pos], temp)); // 왼쪽에서부터 먼저 계산하므로 pos 위치를 다음에 바로 계산
    }
}
```